

CryptoChain Analyzer Dashboard

Final Project Report — Hash Functions and Blockchain

Zihao Ying

1 Introduction

This report documents the CryptoChain Analyzer Dashboard, a real-time Python dashboard built with Streamlit that connects to public Bitcoin blockchain APIs and displays live cryptographic metrics.

The dashboard implements all four required modules (M1–M4) and all three optional modules (M5–M7), totalling seven independent pages. Data is sourced from three free public APIs — Blockstream, Mempool.space and Blockchain.info — with a realistic mock-data fallback so every module remains fully usable when the network is unavailable.

2 Cryptographic Metrics

2.1 SHA-256d and Proof of Work (M1, M2)

SHA-256d — double SHA-256 — is Bitcoin's standard cryptographic hash: `SHA256(SHA256(data))`. Bitcoin's Proof-of-Work (PoW) requires miners to find a nonce such that $\text{SHA256d}(\text{header}) \leq \text{target}$. The 80-byte block header contains six little-endian fields:

- version (4 B) — signals which BIP rules apply
- previous block hash (32 B) — SHA-256d of the previous header; the chain link
- Merkle root (32 B) — root of the transaction hash tree
- timestamp (4 B) — Unix epoch in seconds
- bits (4 B) — compact encoding of the PoW target threshold
- nonce (4 B) — miner-controlled counter iterated until the hash is valid

Module M2 parses all six fields and verifies the PoW locally using Python's built-in hashlib — no external cryptographic library is used.

bits → **target decoding**: The `bits` field encodes the target as a compact floating-point number:

```
target = coefficient × 2(8 × (exponent - 3))
```

A valid block hash, interpreted as a 256-bit little-endian integer, must be $\leq$ target. At current difficulty, approximately 78 leading bits must be zero, so the probability of any single hash attempt succeeding is roughly 2^{-78} .

Field	Example value	Meaning
bits (hex)	0x1703A30C	Compact target stored in block header
Exponent byte	0x17 = 23	$\text{target} = \text{coef} \times 2^{(8 \times (23 - 3))} = \text{coef} \times 2^{160}$
Coefficient	0x03A30C	Mantissa of the compact float
Full 256-bit target	0000000000003A30C...00	Hash must be $\leq$ this value
Leading zero bits	~78 / 256	$P(\text{valid hash}) \approx 2^{-(78)}$

2.2 Difficulty and Network Hash Rate (M1, M3)

The difficulty value is derived from the target by comparing it to the genesis block target (bits = 0x1d00ffff):

```
difficulty = genesis_target / current_target
```

The estimated network hash rate follows from the expected hashes per block and the 600-second target block time:

```
hashrate ≈ difficulty × 232 / 600 [H/s]
```

Bitcoin adjusts difficulty every 2,016 blocks (≈ 14 days) so that the average inter-block time stays close to 600 seconds:

```
new_difficulty = old_difficulty × (actual time for 2016 blocks / 1,209,600 s)
```

The ratio is clamped to [¼, 4] to prevent extreme swings. Module M3 charts this history on a log scale, marks each adjustment event, and plots the block-time ratio per epoch.

2.3 Merkle Trees (M2, M5)

Every block header contains a Merkle root — the root of a binary hash tree built from all transaction IDs. Each internal node is:

```
SHA256d(left_child_bytes || right_child_bytes)
```

When the number of nodes at a level is odd, Bitcoin duplicates the last node before hashing pairs (the odd-node rule). Module M5 reconstructs the full Merkle tree from a live block's transaction list and generates the inclusion proof for any user-selected transaction, showing every SHA-256d step explicitly. The recomputed root is verified against the block header.

2.4 51% Attack Security Model (M6)

Cost model: The real-time network hash rate is derived from the current difficulty. Using NiceHash SHA-256 rental pricing (~\$0.0654/TH/hr), the cost to rent >50% of hash rate exceeds **\$18 million per hour** at current hash rate (~554 EH/s).

Nakamoto (2008) §11 formula: The probability that an attacker controlling fraction q overtakes the honest chain (which is z blocks ahead) is:

$$P(z, q) = 1 - \sum_{k=0}^z \text{Poisson}(k; \lambda) \times (1 - (q/p)^{(z-k+1)})$$

where $\lambda = z \times (q/p)$ and $p = 1 - q$. With $q = 30\%$ and $z = 6$ confirmations: **$P \approx 5.3\%$** . The simplified approximation $(q/p)^z$ gives ≈ 5.2 — very close for small z .

3 AI Component

3.1 M4 — Transaction Fee Estimator (Gradient Boosting Regressor)

Problem: The Bitcoin mempool operates as a fee-priority queue. Miners include transactions in descending order of fee rate (sat/vByte). Predicting the clearing fee for the next block lets users balance confirmation speed against cost.

Model choice — Gradient Boosting Regressor (scikit-learn). Chosen over linear regression, LSTM and Prophet because:

- Fees spike non-linearly when blocks fill — tree-based models handle this without manual feature transforms.
- Robust to outliers from sudden congestion events or whale transactions.
- No feature scaling required.
- Feature importances are interpretable and pedagogically useful.

Features:

Feature	Description
hour	UTC hour of block timestamp — captures intra-day demand peaks
day_of_week	Weekday (0=Mon) — weekly demand cycles (lower at weekends)
tx_count	Number of transactions in the block
size_mb	Block size in megabytes
fullness	size_mb / 1.75 — fraction of the SegWit soft block weight limit
lag_fee	Previous block's median fee rate — momentum / autocorrelation
lag_fee2	Two-block lag — second-order fee autocorrelation
roll_mean	5-block rolling mean — smoothed trend signal

Training: Temporal 80/20 train/test split (no shuffle, to prevent look-ahead bias), plus 3-fold TimeSeriesSplit cross-validation for a robust MAE estimate.

3.2 M4 — Evaluation Results

Metric	Value	Interpretation
MAE	~8 sat/vByte	Primary metric — average absolute error, same unit as target
RMSE	~12 sat/vByte	Penalises large congestion-spike misses more than MAE
MAPE	~19%	Scale-independent percentage error
R ²	~0.10	Fraction of fee variance explained by block-level features
CV-MAE (3-fold)	~10 ±3 sat/vB	TimeSeriesSplit cross-validated MAE — more robust estimate

Interpretation: The low R² is expected. Fee markets are primarily driven by real-time mempool depth — the current backlog of unconfirmed transactions — which is not fully captured by block-level aggregate features. An MAE of ~8 sat/vByte is nonetheless acceptable for priority-tier estimation (LOW / MEDIUM / HIGH / PRIORITY), where adjacent tiers are separated by tens of sat/vByte.

Feature importance: The most important feature is `lag_fee` (previous block median fee), followed by `roll_mean` and `fullness`. Time-of-day (`hour`) ranks fourth, confirming real intra-day demand cycles in the data.

3.3 M7 — Block Anomaly Detector (second AI approach)

Module M7 implements a complementary, unsupervised approach. Under Bitcoin's Poisson mining model, inter-block times T follow $\text{Exp}(\lambda = 1/600 \text{ s})$. Blocks in the far tail may indicate hash rate surges, mining pool bursts, or network partitions.

Two independent detectors per block:

- **Two-sided p-value** against the $\text{Exp}(\lambda=1/600)$ CDF: $p = 2 \times \min(F(t), 1-F(t))$. Flagged if $p < \alpha$ (default 0.05).
- **Z-score on log-transformed inter-arrival times:** $\log(T)$ is approximately Gumbel-distributed; $|z| > \text{threshold}$ (default 2.5) flags an anomaly.

Comparison with M4:

Dimension	M4 · Fee Estimator → M7 · Anomaly Detector
Problem type	Supervised regression → Unsupervised statistical detection
Model	Gradient Boosting Regressor → Exponential distribution baseline
Training data	8-feature block matrix → Inter-arrival times only
Output	Continuous fee value (sat/vByte) → Binary flag + fast/slow direction
Evaluation	MAE, RMSE, MAPE, R^2 → P-value, Z-score, detector agreement F1
Key assumption	Fee predictable from metadata → Block times $\sim \text{Exp}(1/600)$

4 Technical Implementation

The dashboard is built with Streamlit using a sidebar navigation structure with seven pages. Charts are rendered with Plotly. The project has no external cryptographic dependencies — SHA-256d uses only Python's built-in hashlib and struct modules.

Component	Details
APIs	Blockstream (block data, headers, txids), Mempool.space (fee-annotated blocks), Blockchain.info (difficulty history, BTC/USD price)
Machine Learning	scikit-learn GradientBoostingRegressor, TimeSeriesSplit; scipy.stats Exponential distribution
Frontend	Streamlit, Plotly graph_objects, custom dark-theme CSS with IBM Plex fonts
Cryptography	hashlib.sha256 (SHA-256d), struct (little-endian header parsing) — no external crypto library
Fallback	Every API function has a mock_* counterpart; all modules work fully offline

5 References

- [1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. <https://bitcoin.org/bitcoin.pdf>. Cited for §6 (fee incentives) and §11 (attack probability).
- [2] Blockstream. *Esplora REST API Documentation*. <https://github.com/Blockstream/esplora/blob/master/API.md>. Used for block headers, transactions, recent blocks (M1, M2, M5, M7).
- [3] Mempool.space. *API Documentation*. <https://mempool.space/docs/api>. Used for block fee statistics and live recommendations (M4).
- [4] Blockchain.com. *Charts API — Difficulty and Hash Rate*. <https://www.blockchain.com/explorer/charts>. Used for historical difficulty and BTC/USD price (M3, M6).
- [5] Scikit-learn developers. *Gradient Boosting Regressor*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html>. M4 model implementation.